

ESPECIALIZACIÓN EN **INTELIGENCIA DE DATOS** para la Gestión Estratégica

LIMPIEZA, CODIFICACIÓN, NORMALIZACIÓN Y GRÁFICOS

Clase 3 · Taller T8001 — Pre-procesado de datos

Docente: Dr. Lautaro Di Bartolo

Asignatura: T8001 — Pre-procesado de datos

Cohorte o comisión: Cohorte 2026

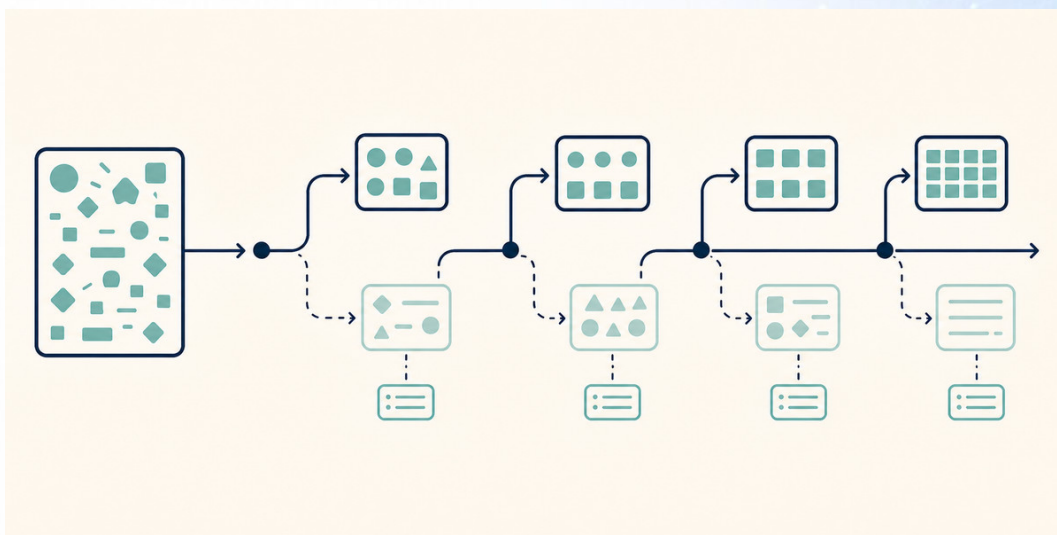


Figura 1. Limpiar no es una operación: es una secuencia de decisiones, y cada una se justifica.

1. INTRODUCCIÓN

1.1 Lo que la clase 2 dejó anotado

La clase 2 terminó con una lista de problemas y no tocó ninguno. Eso fue a propósito: primero se diagnostica y después se decide.

Ahora hay que decidir. Y acá aparece lo que hace difícil esta clase, que no es el código:

Casi ninguna operación de limpieza tiene una única respuesta correcta. Lo que se evalúa no es la operación: es la justificación.

Borrar una fila con un faltante y rellenarla con el promedio son las dos defendibles, y dan resultados distintos. Un ingreso de 620.000 puede ser un error de carga o el hogar más rico del relevamiento. Un `id` repetido puede ser un duplicado o dos visitas al mismo hogar. En los tres casos el trabajo es el mismo: mirar el dato, elegir, y dejar escrito por qué.

1.2 Lo que se supone de las clases anteriores

De la clase 1: que un CSV es texto plano, que la extensión no garantiza nada, que un formato con esquema conserva los tipos y uno de texto plano no, y que el archivo original no se modifica nunca.

De la clase 2: los seis pasos del perfilado, la diferencia entre el tipo de pandas y la escala de medición, las ocho columnas del diccionario de variables, y el filtro de `True` y `False` que elige filas.

La herramienta nueva de esta clase es una sola: `matplotlib`, la librería de gráficos. Aparece en la sección 7 y no antes.

1.3 El relevamiento completo

Hasta ahora trabajamos con doce hogares. El relevamiento entero tiene **cuarenta filas**, y las doce que ya conocés son las doce primeras.

Las veintiocho filas nuevas traen más de los mismos problemas y cinco que con doce filas no aparecían: un acento que falta, una localidad escrita entera en mayúsculas, cuatro fechas en otro formato, dos celdas de servicios con otro separador y un ingreso de cero. Y traen las dos cosas que con doce filas no se podían ver: ahora hay duplicados, y hay suficientes ingresos para que dos métodos de detección de valores extremos den respuestas distintas.

Los tres números de esta clase:

Cuarenta filas. Treinta y nueve cuando saquemos el duplicado exacto. Treinta y siete hogares cuando resolvamos el id repetido.

Esos tres números no son lo mismo, y la diferencia entre el segundo y el tercero es una decisión tuya.

2. EL ORDEN DE LA LIMPIEZA

2.1 El mismo dato, los mismos pasos, distinto resultado

Las operaciones de limpieza no son independientes. El resultado depende del orden en que se hacen, y eso no es un detalle de implementación: cambia los números del informe.

Un caso concreto de nuestra tabla. Hay diez filas donde `cobertura_salud` y `satisfaccion` dicen NULL, y hay una fila repetida entera. Si informás el porcentaje de faltantes antes de sacar el duplicado, sobre cuarenta filas, te da 25%. Sobre las treinta y siete que quedan al final, es 27%. Los dos números son ciertos y no dicen lo mismo, así que hay que decir sobre qué tabla están calculados.

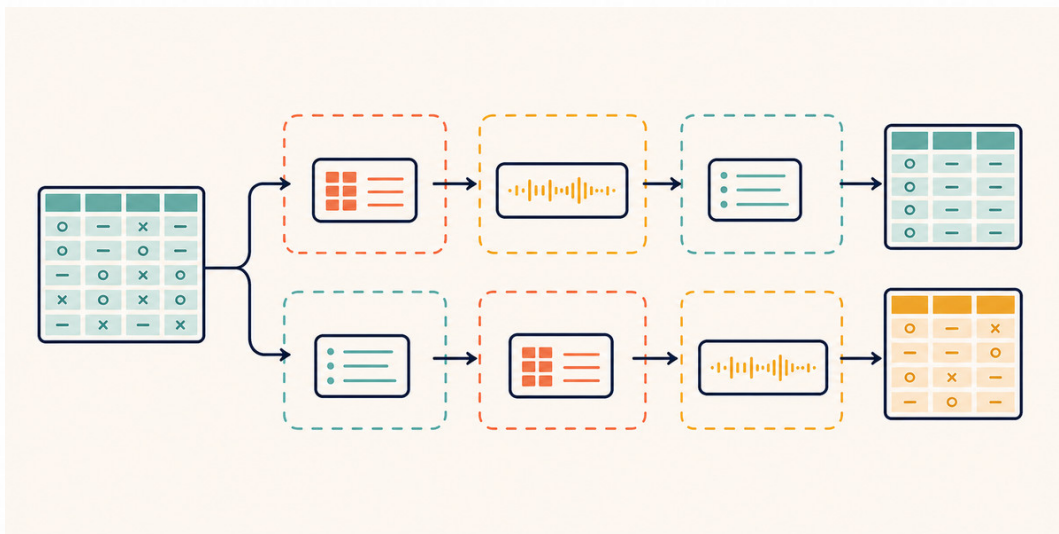


Figura 2. Las mismas operaciones, en otro orden, no dan la misma tabla.

Otro caso, más caro. Si un hogar estuviera cargado dos veces, uno como Ramallo y otro como ramallo, buscar duplicados **antes** de unificar el texto no lo reconocería: para la máquina son dos textos distintos. Unificando primero, sí. En nuestra tabla ese par no está, y por eso hay que saber que puede estar.

No hay un orden único correcto, pero hay uno que funciona casi siempre:

1. Unificar lo que se escribió de varias formas: texto y categorías.
2. Sacar lo que sobra: duplicados exactos y conflictos.
3. Convertir a su tipo: fechas y números.
4. Decidir sobre lo que falta: faltantes y centinelas.
5. Recién entonces: valores extremos, codificación y normalización.

El orden de este apunte no es ese, y es a propósito: acá los temas van de más simple a más difícil, para poder explicarlos. La lista de arriba es el orden de ejecución; los títulos que siguen no lo respetan.

2.2 El pipeline reproducible

Un **pipeline** es la secuencia completa de operaciones, escrita en un solo lugar y en orden, de manera que se pueda ejecutar de nuevo sobre el archivo original y dar exactamente el mismo resultado.

No es una herramienta ni una librería: es una forma de escribir. En la práctica son dos reglas.

La primera es que el pipeline arranca leyendo el crudo, siempre. Nunca una tabla intermedia que quedó en memoria de una ejecución anterior.

La segunda es que cada paso deja el anterior intacto. Si el paso 3 se equivocó, se corrige el paso 3 y se vuelve a ejecutar todo, sin deshacer nada a mano.

Si no podés borrar todo lo que generaste y reconstruirlo ejecutando de arriba hacia abajo, no tenés un pipeline: tenés una tabla que nadie sabe cómo se armó.

2.3 La regla del crudo, otra vez

Es la regla de la clase 1, sección 3.8, y acá es donde más cuesta respetarla, porque todo el trabajo de esta clase modifica el dato.

La forma de respetarla es no modificar nunca la tabla que leíste: los cambios se hacen sobre una copia con otro nombre, y el crudo queda disponible para volver a leerlo. En la clase 2 eso salvó el diagnóstico: los ocho NULL de esa clase solo se vieron releendo el archivo sin interpretar.

3. DUPLICADOS

3.1 Exactos y aproximados

Un **duplicado exacto** es una fila idéntica a otra en todas sus columnas. Se detecta y se saca en una línea, y casi siempre es un error de carga.

Nuestra tabla tiene uno: la última fila repite entera la anterior. Cuarenta filas pasan a treinta y nueve, en una línea:

```
sin_duplicados = df.drop_duplicates()
```

Un **duplicado aproximado** es la misma observación cargada dos veces con alguna diferencia: una mayúscula, un espacio, un acento. Esos no los encuentra ninguna función, porque para la máquina son dos filas distintas.

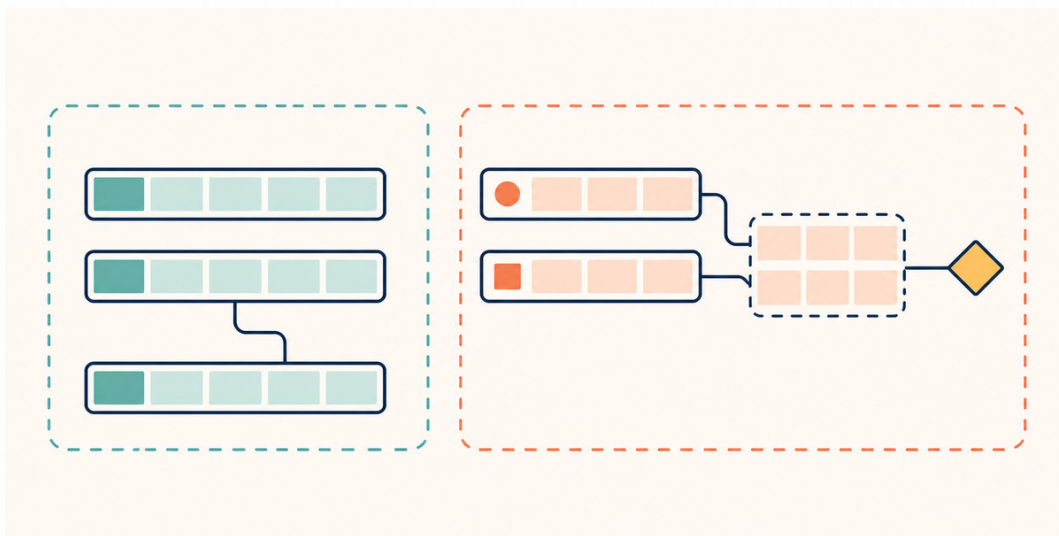


Figura 3. El duplicado exacto se cae solo. El aproximado hay que ir a buscarlo.

De ahí sale la consecuencia de orden de la sección 2.1: **el texto se unifica antes de buscar duplicados**, no después. Si no, los aproximados sobreviven.

3.2 Cuando un repetido es un dato legítimo

Antes de borrar, la pregunta es si las dos filas **tenían** que ser distintas.

Dos visitas al mismo hogar el mismo día son dos observaciones legítimas, y en nuestra tabla no hay nada que las distinga. Si el relevamiento no guardó la hora ni el nombre del encuestador, las dos filas son idénticas, y borrar una pierde información real.

La decisión se toma con el diccionario de variables, no con el archivo. Si el diccionario dice que **id** identifica al hogar y que hay una fila por hogar, entonces una fila repetida es un error. Si dice que hay una fila por visita, no lo es.

3.3 El id repetido no es un duplicado: es un conflicto

Después de sacar el duplicado exacto quedan treinta y nueve filas, y todavía queda un problema: el id 41 aparece dos veces, con contenido distinto.

id	localidad	personas	ingreso	satisfaccion
41	Córdoba	3	213000	regular
41	Córdoba	5	254000	buena

Esto no se resuelve borrando. Son dos filas que dicen ser el mismo hogar y no coinciden, así que una de las dos está mal, o son dos hogares a los que se les asignó el mismo número.

Hay tres salidas posibles, y ninguna es gratis. Se puede volver a la fuente y preguntar, que es la única que resuelve de verdad. Se puede quedar con una de las dos y anotar el criterio, por ejemplo la última cargada. O se pueden descartar las dos, porque no se sabe cuál vale.

Con lo que tenemos elegimos la tercera y la dejamos escrita: dos filas en conflicto sin forma de desempatarlas se van. Son dos filas, pero **un solo hogar**, y el relevamiento queda en **treinta y siete hogares**.

4. FALTANTES

4.1 El cero, el vacío y el centinela son tres cosas distintas

En la columna ingreso hay tres celdas que a primera vista dicen lo mismo, y no dicen lo mismo:

- **El hogar 3 tiene el ingreso vacío.** Nadie escribió nada. Es una ausencia.
- **El hogar 38 tiene un 0.** Es un ingreso declarado, y vale cero. No falta: es un dato.
- **El hogar 9 tiene 999999.** Es un **centinela**: un número que alguien eligió para decir "no sé". Es una ausencia disfrazada de dato.

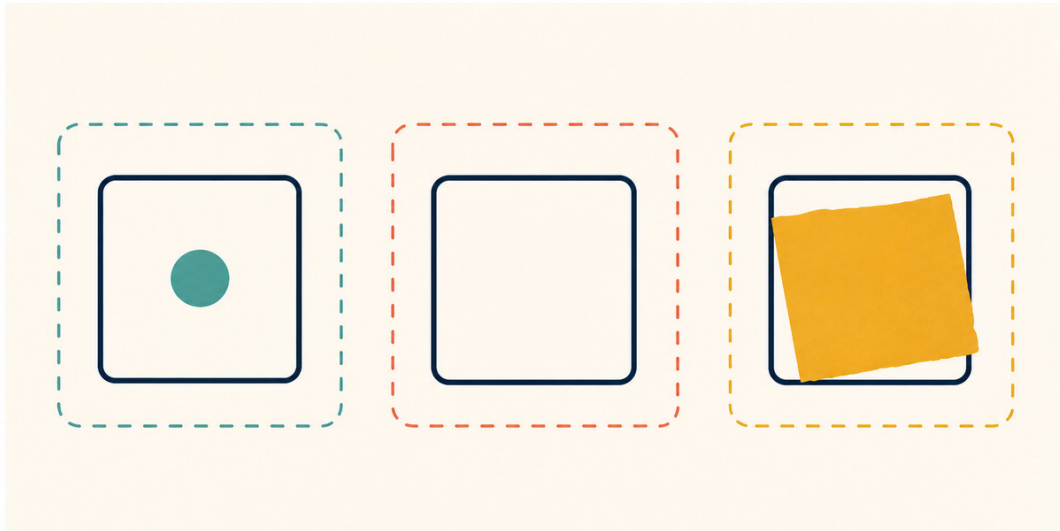


Figura 4. Tres celdas que parecen lo mismo y significan tres cosas distintas.

Las consecuencias son inmediatas y van en direcciones opuestas. El 0 baja el promedio y **tiene que bajarlo**, porque es un ingreso real. El 999999 lo sube y **no tiene que subirlo**, porque no es un ingreso. Y el vacío no hace nada, que es lo correcto.

El primer paso con los faltantes no es decidir qué hacer: es averiguar cuáles son faltantes.

Al centinela se lo convierte en ausencia al leer, y al cero no se lo toca:

```
df = pd.read_csv(ruta, na_values={"ingreso": [999999]}) # el 0 no se toca
```

4.2 Borrar o imputar

Con los faltantes de verdad hay dos caminos, y los dos son defendibles.

Borrar la fila entera es honesto y no inventa nada. El costo es que pierde las otras siete columnas de esa fila, que estaban completas. Y si los faltantes no están repartidos al azar, borrar sesga la muestra: es exactamente lo que vimos en la clase 2, sección 3.2.

Imputar es rellenar el hueco con un valor estimado: el promedio, la mediana, el valor más frecuente, o algo más elaborado. Conserva la fila y agrega un dato que nadie midió. Si después alguien calcula la variabilidad de esa columna, la va a encontrar más chica de lo que es, porque los valores imputados no varían.

La decisión depende de tres cosas: si la columna es central para tu análisis, si los faltantes están repartidos o agrupados, y por qué faltan. Son las tres preguntas de la clase 1, sección 4.2.

Con pocos faltantes repartidos al azar en una columna secundaria, borrar. Con muchos, o agrupados, o en la columna principal, ninguna de las dos alcanza: hay que ir a la fuente.

Y si el camino elegido es imputar, vale una regla más: **dejá una columna que diga cuáles imputaste**. Si no, el que lee tu tabla no puede distinguir un dato medido de uno inventado.

4.3 Los tres mecanismos

Un faltante no es solo un hueco: tiene una causa, y la causa decide si podés hacer algo con él. La clasificación estándar, la de [Rubin \(1976\)](#), tiene tres casos.

- **Al azar del todo.** El faltante no depende de nada. Se cayó una fila al exportar. Es el único caso en el que borrar no sesga nada.
- **Al azar condicionado.** El faltante depende de otra columna que sí tenés. Si falta el ingreso sobre todo en una localidad, y la localidad está en la tabla, se puede imputar por localidad y la estimación mejora.
- **No al azar.** El faltante depende del valor que falta. Los hogares de ingreso más alto son los que menos declaran su ingreso. Este es el caso grave: cualquier cosa que hagas sesga el resultado, y ninguna técnica lo arregla.

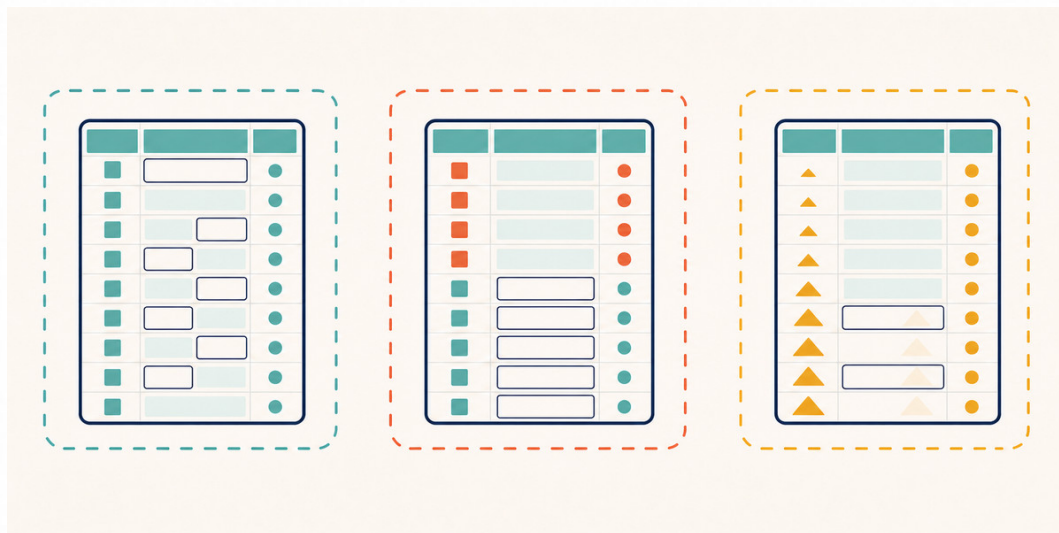


Figura 5. De qué depende la ausencia decide qué se puede hacer con ella.

Lo incómodo es que **el tercer caso no se puede distinguir de los otros dos mirando la tabla**, por la misma razón por la que el sesgo no se ve: el dato que necesitás para comprobarlo es justamente el que falta. Se descarta o se sospecha con lo que sepas del instrumento, que es el trabajo de la clase 2.

4.4 Qué hacemos con las diez filas con NULL

Diez de las treinta y siete filas que quedaron dicen NULL en `cobertura_salud` y en `satisfaccion`, siempre en las mismas filas. La clase 2 llegó hasta acá y se detuvo: cuatro explicaciones posibles, todas sobre el instrumento, y ninguna forma de elegir sin el cuestionario.

Ahora hay que decidir igual. Y la decisión depende de cuál de las explicaciones supongas:

- Si fue un **salto condicional**, esas dos preguntas no correspondían a esos hogares. Entonces no son faltantes: son "no aplica", y borrar esas diez filas sería borrar diez hogares válidos que no usan el centro de salud.
- Si fue un **fallo del instrumento**, sí son faltantes.

Las dos lecturas llevan a decisiones opuestas, y son diez hogares de treinta y siete.

Lo que hacemos, y queda escrito: **no borramos nada y no imputamos nada**. Las dos columnas se conservan con su ausencia marcada, y todo análisis que las use informa que se calcula sobre veintisiete hogares y no sobre treinta y siete. Es la única salida que no inventa un dato ni descarta uno bueno.

Cuando no se puede decidir, la opción correcta es no decidir y decirlo. Lo que no se puede es decidir en silencio.

5. TEXTO, FECHAS Y VALORES EXTREMOS

5.1 Los tres pasos del texto

localidad tiene **siete formas de escribir tres localidades**. Así queda en las treinta y siete filas que sobrevivieron a la sección 3:

Valor	Filas
Ramallo	12
Córdoba	10
Concepción	9
ramallo	2
CONCEPCIÓN	2
Ramallo	1
Cordoba	1

Las cuatro últimas son el mismo problema que diagnosticó la clase 2: un espacio final que no se dibuja, una minúscula, una palabra entera en mayúsculas y un acento que falta. Se arreglan en tres pasos, y en este orden.

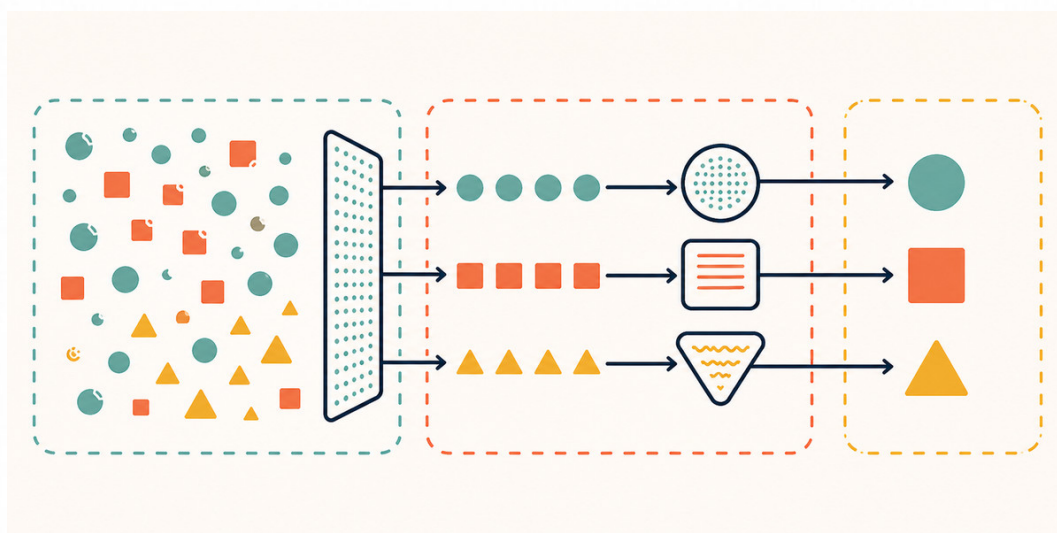


Figura 6. Cada paso junta lo que estaba separado por una diferencia de escritura.

Paso 1: sacar los espacios de los bordes. Siete formas pasan a seis. Es el paso que más rinde por línea escrita, porque el defecto que arregla es invisible.

Paso 2: pasar todo a minúscula. Seis pasan a cuatro. Se juntan Ramallo con ramallo y Concepción con CONCEPCIÓN.

Paso 3: sacar los acentos. Cuatro pasan a tres, porque córdoba y cordoba eran la misma.

Los tres pasos son tres llamadas encadenadas:

```
limpia = (df["localidad"].str.strip()          # paso 1: los bordes
          .str.lower()                        # paso 2: la minúscula
          .str.normalize("NFKD")             # paso 3: separa el acento
          .str.encode("ascii", "ignore").str.decode("ascii"))
```

Los tres pasos son **mecánicos**: los aplicás sin mirar el contenido y no pueden estar mal. Ahí termina lo automático.

Después queda un cuarto paso que ninguna función hace: decidir si Concep. y Concepción son la misma localidad. Eso no es una diferencia de escritura, es una abreviatura, y la herramienta no puede saberlo. Existen funciones que buscan textos parecidos y **sugieren** candidatos, y esa palabra es la importante: sugieren.

Los tres pasos mecánicos los hace la herramienta. El cuarto lo hacés vos, mirando la lista de valores únicos, y lo dejás escrito como un mapa de equivalencias.

Que la unificación pase de siete valores a tres tiene una consecuencia que se ve más adelante: en la sección 6.1, codificar `localidad` sin limpiarla antes produce siete columnas en lugar de tres.

5.2 Fechas con formatos mezclados

`fecha_visita` llega como texto. De las treinta y siete filas, treinta y tres están en formato ISO 8601, 2025-03-14, y **cuatro están en dd/mm/aaaa**, como 25/03/2025. Es lo que produce un formulario que no fijó el formato, y es el caso de la clase 1, sección 5.2.

Si convertís la columna sin decir nada, pandas se planta con un `ValueError` y te dice qué valor no encajó. Si le pedís `errors="coerce"`, convierte las treinta y tres que entiende y deja las otras cuatro sin convertir. Las dos salidas son buenas: **el problema se ve**.

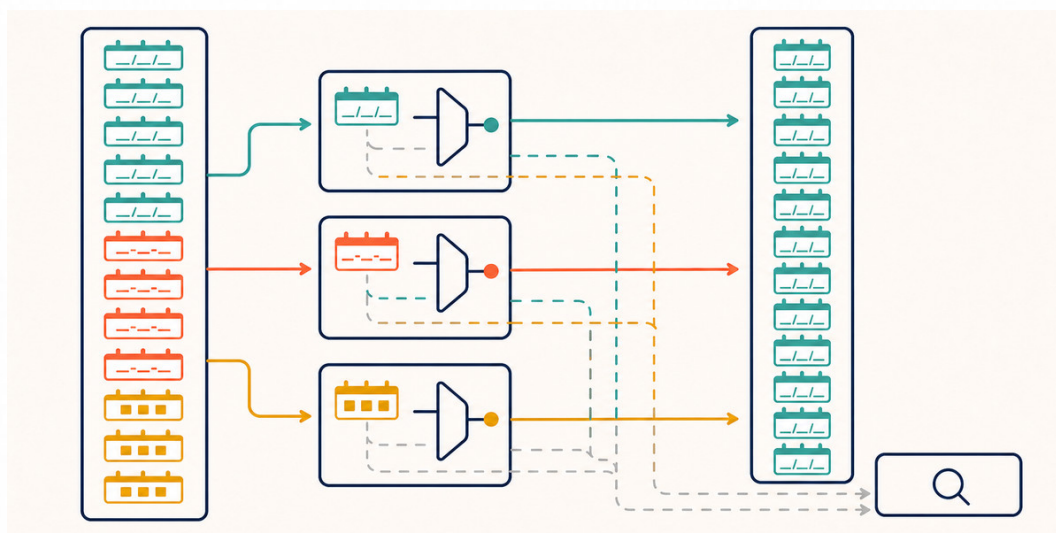


Figura 7. La conversión parcial es una buena noticia: señala exactamente qué filas revisar.

Lo que sigue es la trampa de esta sección. Existe un argumento que le dice a pandas que acepte formatos mezclados y adivine el de cada fecha por separado: `format="mixed"`. Con eso **la columna entera se convierte, sin un solo error**. Parece la solución.

No lo es. Donde una fecha admite dos lecturas, pandas elige mes/día, y en nuestra tabla **corrompe dos de las treinta y siete fechas**: `02/04/2025` se convierte en el 4 de febrero y `07/04/2025` en el 4 de julio. El relevamiento pasa a ir del 4 de febrero al 4 de julio, cuando en realidad duró veintiséis días entre el 14 de marzo y el 9 de abril.

Y hay algo peor. Mirar el mínimo y el máximo avisa que algo está mal, y no dice qué filas son. Para encontrarlas hay que filtrar. Acá el filtro natural, quedarse con lo que cae fuera de marzo y de abril, encuentra las dos. Pero no siempre alcanza: una fecha como `03/04/2025` se convertiría en el 4 de marzo, que cae en un mes del relevamiento, y ni el rango ni ese filtro la delatarían. Mirar el rango es necesario, y no es una prueba.

Una conversión que no falla no es una conversión correcta. Después de convertir fechas, mirá el mínimo y el máximo: si el rango no es el que esperabas, están mal.

Y antes de declarar el formato, una pregunta que la herramienta no hace. De las cuatro, `25/03/2025` y `28/03/2025` no tienen otra lectura: no hay mes 25, y por eso `format="mixed"` las salvó solas. Pero `02/04/2025` puede ser el 2 de abril o el 4 de febrero. Del texto solo, no sale. Sale de la tabla: las filas vienen en orden de visita, y esa fila cae entre una del 2 y una del 3 de abril. Con eso `dd/mm/aaaa` queda confirmado. Sin eso, el formato es un supuesto que hay que escribir.

La forma correcta son dos pasos, y no tiene nada de elegante. Primero se convierte lo que está en ISO, y las cuatro que no lo están quedan sin convertir. Después se convierten esas cuatro, declarando su formato:

```
fechas = pd.to_datetime(df["fecha_visita"], format="ISO8601", errors="coerce")
faltan = df.loc[fechas.isna(), "fecha_visita"]
fechas = fechas.fillna(pd.to_datetime(faltan, format="%d/%m/%Y"))
```

Treinta y siete de treinta y siete, y el rango va del 14 de marzo al 9 de abril, que es el que tiene que ser.

5.3 Un extremo no es un error

Un **valor extremo** es un valor muy alejado del resto. Y eso es todo lo que quiere decir: no quiere decir que esté mal.

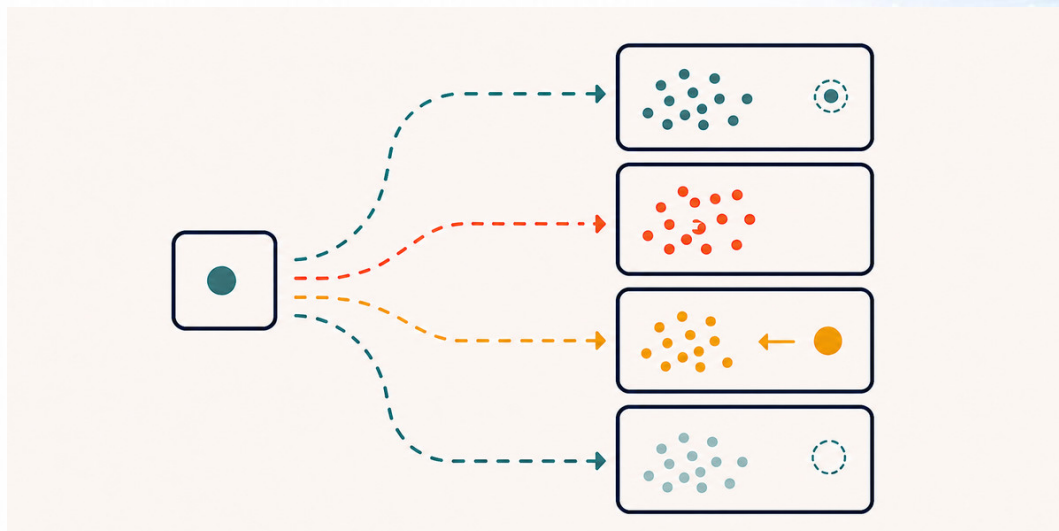


Figura 8. La distancia al resto no dice si el valor es falso.

Nuestra tabla tiene tres, y son tres casos distintos:

- **El 999999 del hogar 9.** No es un extremo: es un centinela, y ya lo tratamos en la sección 4.1. Los centinelas se sacan **antes** de buscar extremos, porque si no, dominan el cálculo.
- **El 620000 del hogar 39.** Es tres veces el promedio. Puede ser un error de tipeo, un cero de más sobre 62.000, o el hogar de mayor ingreso del relevamiento.
- **Las 12 personas del hogar 39.** El mismo hogar. Un hogar de doce personas existe, y que sea el mismo hogar que tiene el ingreso más alto hace la fila más creíble, no menos: un hogar grande con varios adultos que aportan.

Ese último razonamiento es el que hay que hacer, y no lo hace ninguna función: **cruzar el valor extremo con las otras columnas de su fila**. Un ingreso de 620.000 en un hogar de una persona es sospechoso. En un hogar de doce, es plausible.

Y conviene desconfiar de la lectura fácil en las dos direcciones, porque una fila que es la más extrema en dos columnas a la vez también es el patrón de un registro mal cargado. Lo que resuelve el empate es una cuenta: 620.000 dividido doce personas son 51.667 pesos por persona, que es exactamente la mediana del relevamiento. Ese hogar no es raro en ingreso por persona, es raro en tamaño, y su ingreso es el que le corresponde a su tamaño. Ahí sí la fila se sostiene.

Un extremo no se borra: se investiga. Y si no se puede investigar, se deja y se anota, o se saca y se anota. Lo que no se hace es sacarlo en silencio porque afeaba el gráfico.

5.4 IQR y z-score sobre la misma variable

Hay dos formas habituales de marcar candidatos, y la lección de esta sección es que **no dan la misma respuesta**.

El **primer cuartil** es el valor que parte el cuarto de abajo: de los treinta y cinco ingresos ordenados, nueve quedan por debajo de 173.500. El tercero hace lo mismo con el cuarto de arriba: nueve quedan por encima de 219.500. Entre los dos está la mitad del medio.

El **IQR** es la distancia entre esos dos cuartiles, y la regla de marcar lo que se aleja una vez y media viene de [*Exploratory Data Analysis* \(Tukey, 1977\)](#). Marca lo que está más de una vez y media ese rango por debajo del primer cuartil o por encima del tercero. No mira cuánto vale un valor extremo: mira en qué puesto quedó. Si cambiaras el 620.000 por 5.000.000, los cuartiles y los dos límites quedarían exactamente iguales.

El **desvío estándar** mide cuánto se aleja del promedio un valor típico: con un promedio de 203.471 y un desvío de 87.360, treinta y dos de los treinta y cinco hogares están a menos de 87.360 pesos del promedio.

El **z-score** mide a cuántos desvíos está cada valor: se resta el promedio y se divide por el desvío. Para el hogar 39: 620.000 menos 203.471 son 416.529, dividido 87.360 da 4,77. Usa el promedio y el desvío, y los dos se corren cuando entra un valor muy grande. La consecuencia es que **un extremo puede tapar a otro**.

Sobre los treinta y cinco ingresos declarados, el primer cuartil da 173.500 y el tercero 219.500, o sea un IQR de 46.000. El límite de arriba queda en 288.500:

```
q1, q3 = df["ingreso"].quantile([0.25, 0.75])
iqr = q3 - q1
extremos = df[(df["ingreso"] < q1 - 1.5 * iqr) | (df["ingreso"] > q3 + 1.5 * iqr)]
```

Valor	Hogar	IQR	z-score
0	38	lo marca	lo marca, z = -2,33
310.000	7	lo marca	no lo marca, z = 1,22
620.000	39	lo marca	lo marca, z = 4,77

Los dos coinciden en los dos extremos y **discrepan en el 310.000 del hogar 7**, que es un hogar que conocés desde la clase 2. El z-score no lo marca porque el 620.000 le agrandó el desvío hasta 87.360. Si sacaras el 620.000, el 310.000 daría 2,40 y quedaría marcado. El 620.000 también se atenúa a sí mismo, pero con 4,77 no alcanza a esconderse.

Dos aclaraciones sobre los umbrales, porque los dos son convenciones y no leyes. El 1,5 del IQR es la regla habitual y viene de la práctica, no de una demostración. Y el umbral del z-score que se usa normalmente es 3, no 2: con 2 marcás alrededor del 4,5% de cualquier columna, incluso de una perfectamente limpia. Acá usamos 2 para que el ejemplo tenga con qué discrepar, y ese es todo el motivo.

Con personas no hay discrepancia: los dos métodos marcan el 12 y nada más.

Cuando dos métodos discrepan, el que decide no es el método: sos vos. Y lo que se informa es cuál usaste y por qué, no el resultado solo.

Una última cosa sobre el 0 del hogar 38: los dos métodos lo marcan, y **no es un error**. Es un ingreso declarado de cero, que ya decidimos conservar en la sección 4.1. Es el mejor ejemplo de por qué la lista que devuelven estos métodos se llama de candidatos y no de errores.

6. CODIFICACIÓN Y NORMALIZACIÓN

La palabra normalización significa tres cosas distintas según quién la use, y conviene separarlas antes de empezar.

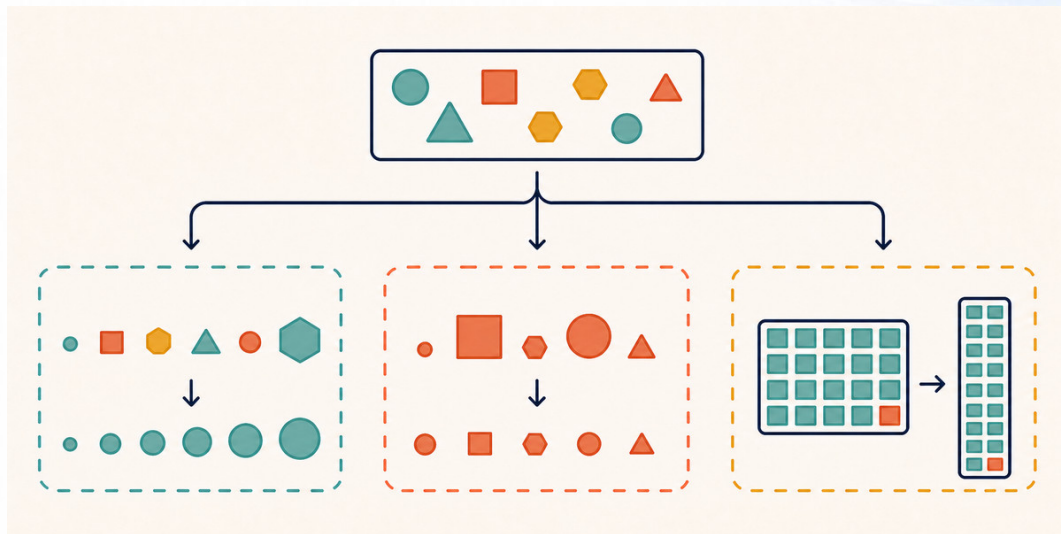


Figura 9. Tres operaciones distintas con el mismo nombre.

Codificar es convertir categorías en números. **Escalar** es llevar variables numéricas a un rango comparable. Y **normalizar la estructura** es cambiar la forma de la tabla, sin tocar ningún valor. Las tres se hacen al final, cuando el dato ya está limpio.

6.1 Codificar una ordinal y una nominal

Las dos se convierten a número, y de dos maneras distintas, porque la escala manda. Es la sección 6.2 de la clase 2, aplicada.

satisfacción es ordinal, así que el número tiene que respetar el orden: mala es 1, regular es 2 y buena es 3. Una sola columna. Las treinta y siete filas se reparten en 11 regulares, 9 buenas y 7 malas, más las diez que están vacías.

Ese número sirve para ordenar y para sacar la mediana, que da 2, o sea regular. **No sirve para promediar**, y la razón es que el 1, el 2 y el 3 los elegiste vos: eligiéndolos supusiste que de mala a regular hay la misma distancia que de regular a buena, y la escala no dice eso. La prueba es que si codificás 1, 2 y 10, la mediana sigue siendo regular y el promedio salta de 2,07 a 4,4. Un resumen que cambia cuando cambiás una etiqueta no resume el dato: resume tu elección.

Esto es lo que la clase 2 dejó anotado en su sección 6.3, cuando dijo que pandas no calcula la mediana de una columna de texto porque no sabe el orden. El orden se declara acá, en el diccionario que traduce cada categoría a su número.

localidad es nominal, así que no hay ningún orden que respetar, y numerarla de 1 a 3 inventaría uno. Lo que se hace es una columna por categoría, con un 1 donde corresponde y un 0 donde no.

Acá se paga la sección 5.1. Con la columna limpia salen **tres** columnas. Con la columna sucia salen **siete**, una por cada forma de escribir, y cuatro de ellas con una o dos filas en 1.

Codificar es lo último. Si codificás antes de unificar el texto, el error se multiplica por la cantidad de categorías falsas.

Y una advertencia que se paga más adelante: las tres columnas suman siempre 1, así que la tercera se deduce de las otras dos. Si van a entrar en un modelo de regresión, se deja una afuera. Para contar frecuencias, las tres sirven.

6.2 Normalización como escalado

Dos variables numéricas del mismo relevamiento pueden estar en rangos que no se pueden comparar: personas va de 1 a 12 y ingreso va de 0 a 620.000. Escalar es llevarlas a un rango común.

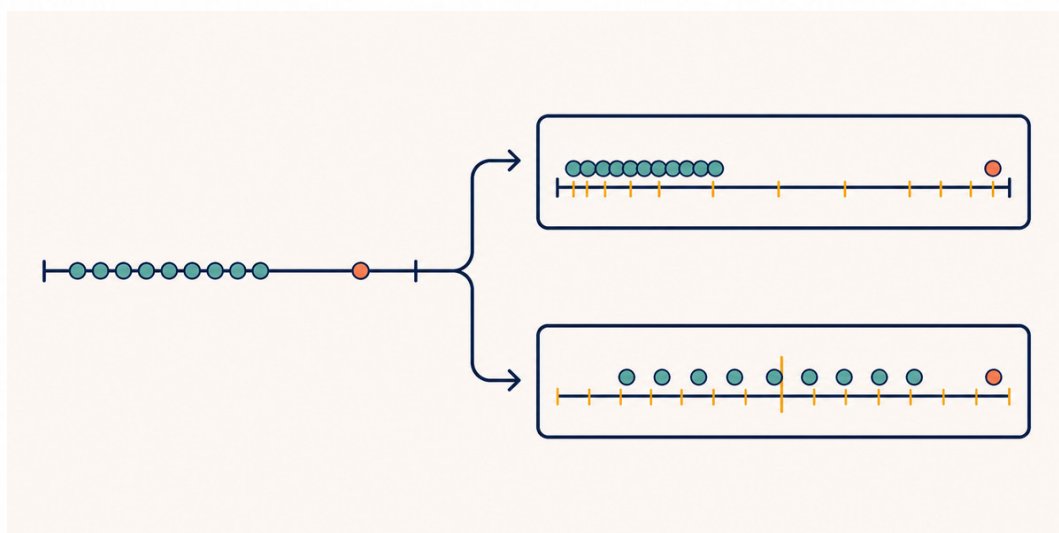


Figura 10. Dos maneras de hacer comparables dos variables que estaban en rangos distintos.

Min-max lleva todo al rango de 0 a 1: el mínimo va a 0, el máximo va a 1 y el resto queda en el medio. Con personas, el 1 va a 0, el 12 va a 1, y el 3, que es el valor más frecuente, va a 0,18. Ese 0,18 es la señal de que el 12 está estirando la escala: la mayoría de los hogares queda apretada en la parte de abajo.

Z-score centra en el promedio y mide en desvíos: el promedio va a 0, y cada valor dice a cuántos desvíos está. El 3 va a $-0,37$ y el 12 va a $3,92$. No queda acotado entre 0 y 1, y eso es una ventaja acá, porque el 3,92 dice sin ambigüedad que ese valor es raro.

La elección es práctica. Min-max cuando necesitás un rango acotado. Z-score cuando te interesa cuán lejos del centro está cada valor. Con extremos, ojo con los dos: el min-max se define con dos observaciones, el mínimo y el máximo, así que un solo error de carga te corre toda la escala; el z-score usa las treinta y siete, así que aguanta más, pero tampoco es inmune, y la sección 5.4 lo muestra.

Ninguna de las dos cambia la forma de la distribución ni arregla un extremo: lo mueven de lugar.

6.3 Normalización como estructura

Esto no toca ningún valor: cambia la forma de la tabla. Es la regla cardinal de la clase 1, sección 3.5, aplicada al revés.

Una tabla **ancha** tiene una columna por categoría. Una tabla **larga**, que es la forma ordenada de la clase 1, tiene una fila por combinación, con una columna que dice cuál es la categoría y otra que dice el valor.

Nuestra tabla no está ni en una forma ni en la otra: tiene una fila por hogar y una columna por variable, que es la forma normal de una tabla de casos. La forma larga aparece cuando querés apilar varias variables en una sola columna de valores. Y cuando codificás *localidad* en tres columnas, te movés hacia el lado ancho, que es el que piden casi todas las herramientas de análisis.

Las dos formas son correctas y sirven para cosas distintas. La larga es la que se guarda y la que crece sin reescribir nada cuando aparece una categoría nueva. La ancha es la que se calcula y la que se grafica.

6.4 Una celda, muchos valores

servicios guarda hasta cuatro servicios en una sola celda. Rompe la regla de un valor por celda, y para contar cuántos hogares tienen gas hay que partirla.

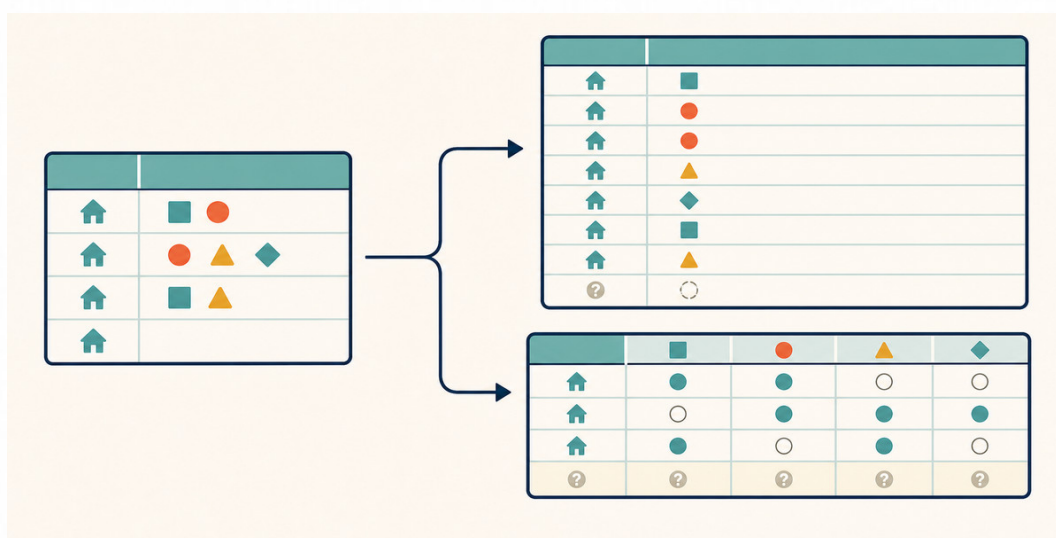


Figura 11. La misma celda se puede abrir a lo largo o a lo ancho, y sirven para cosas distintas.

Hay dos salidas. **A lo largo**, una fila por hogar y por servicio: el hogar 1 pasa a ocupar tres filas. Es la forma que respeta la regla cardinal. **A lo ancho**, una columna por servicio con 1 y 0, que es lo mismo que hicimos con `localidad` en la sección 6.1. Es la forma que se calcula.

Y acá hay un defecto plantado que vale la sección entera. Casi todas las filas separan con punto y coma, `agua;luz;gas`, pero **dos usan coma**, `agua, luz`. Si partís por punto y coma, la función no da ningún error y devuelve **seis** columnas donde tendría que haber cuatro: aparecen dos columnas que se llaman `agua, luz` y `agua, luz, gas`, cada una con una sola fila en 1.

De ahí sale la comprobación: cuando abrís una columna multivalor, contá las columnas que salieron. Si son más que los valores posibles, hay un separador que no viste.

Con la coma reemplazada antes de partir, salen las cuatro que corresponden:

```
tabla = df["servicios"].str.replace(", ", ";").str.get_dummies(sep=";")
```

Luz en 35 hogares, agua en 31, gas en 17 y cloacas en 5.

7. REPRESENTACIONES GRÁFICAS

Después de todo lo anterior, la tabla quedó en **treinta y siete hogares**, de los cuales treinta y cinco declararon su ingreso. Recién ahora se grafica, y por una razón: un gráfico hecho sobre datos sucios muestra los defectos, no los datos.

7.1 La escala elige el gráfico

No es una cuestión de gusto. La escala de medición de la clase 2, sección 6.2, decide qué gráfico tiene sentido:

Si la variable es	El gráfico es	Y el otro está mal porque
Nominal	Barras	Un histograma supone un orden
Ordinal	Barras en el orden de la escala	Ordenar por frecuencia borra el orden
Discreta, pocos valores	Barras, una por valor	Un histograma inventa intervalos
Continua	Histograma o caja	Barras con 35 valores no se leen
Dos numéricas	Dispersión	Dos histogramas no muestran la relación

La fila de la ordinal es la que más se incumple, y nuestra tabla lo muestra. Ordenadas por frecuencia, las respuestas de satisfacción salen regular, buena, mala. En el orden de la escala son mala con 7, regular con 11 y buena con 9.

El primer orden no es solo distinto: **es ilegible**. Con el orden de la escala se ve que la distribución tiene su pico en el medio. Con el orden por frecuencia, eso desaparece.

Una ordinal se grafica en el orden de la escala, siempre, aunque la función ordene por frecuencia por defecto.

7.2 El gráfico como diagnóstico

Un gráfico de diagnóstico no se hace para mostrárselo a nadie. Se hace para ver algo que en la tabla no se ve, y después se descarta.

El mejor ejemplo está en nuestra tabla. Si medís la relación entre personas e ingreso, el **coeficiente de correlación** da 0,79. Ese coeficiente va de -1 a 1: en 0 no hay relación, en 1 cada vez que una variable sube la otra sube en la misma proporción, y en -1 baja. Un 0,79 es una relación fuerte: los hogares más grandes tienen más ingreso.

El gráfico de dispersión cuenta otra cosa. Hay una nube de treinta y cuatro puntos y **un punto solo, arriba a la derecha**: el hogar 39, el de doce personas y 620.000 pesos. Sin ese hogar, el coeficiente baja a 0,56.

O sea: **un hogar de treinta y cinco mueve buena parte del número**. Eso no se ve en el coeficiente y salta a la vista en el gráfico.

Dos advertencias sobre qué concluir. La relación no depende de ese hogar: si en lugar de las magnitudes usás los puestos, el coeficiente pasa de 0,58 a 0,54, o sea casi no se mueve. Lo que el hogar 39 infla no es la asociación, es la estimación.

Y una sobre qué estamos midiendo. Es el ingreso **total** del hogar, que crece con la cantidad de personas que aportan casi por definición. Si en su lugar medís el ingreso por persona, la relación se da vuelta y da $-0,56$: los hogares más grandes tienen menos ingreso por persona. Dos preguntas distintas, dos respuestas opuestas, el mismo dato.

Un número resume y por eso esconde. Antes de creerle a un coeficiente, mirá la nube de puntos.

Lo mismo vale para el histograma y para el diagrama de caja: no sirven para presentar, sirven para encontrar el valor que no encaja, la distribución con dos picos y el rango que no tiene sentido.

7.3 Cómo miente un gráfico

Un gráfico correcto puede mentir. Las dos formas más frecuentes están en nuestra tabla.

El eje que no arranca en cero. Los tres conteos de localidad son 15, 11 y 11: el más alto es 1,36 veces el más bajo. Si el eje vertical arranca en 10 en lugar de en 0, las barras miden 5, 1 y 1, y el más alto **parece cinco veces el más bajo**. Ningún número está mal. La conclusión que saca quien lo mira, sí.

En un gráfico de barras el eje arranca en cero. No es una convención estética: la longitud de la barra es la que codifica la cantidad.

El promedio sin la dispersión. Decir que el ingreso promedio es de 203.000 no es falso, y esconde que hay un hogar en 0 y otro en 620.000. La mediana es 193.000: los diez mil de diferencia son ese hogar de 620.000 tirando el promedio para arriba. Un diagrama de caja o un histograma al lado del número muestran lo que el número tapa.

Y el alcance de esa regla: es de las barras, no de todos los gráficos. En una línea o en una nube de puntos la posición codifica la cantidad, así que ahí cortar el eje es legítimo, y forzar el cero muchas veces aplasta lo que querías mostrar.

Hay una tercera forma de mentir, y es la que más aparece en informes reales: **el extremo que se saca para que el gráfico quede lindo**. Si sacás el hogar 39 del gráfico de dispersión sin decirlo, el gráfico queda más prolijo y la relación que muestra es otra. Eso no es limpiar: es elegir el resultado.

7.4 Lista de verificación

Cuando tengas que limpiar un conjunto de datos ya diagnosticado, en este orden:

1. **Ejecutá el pipeline desde el crudo, siempre.** Si un paso se corrige, se corre todo de nuevo.
2. **Unificá el texto antes de buscar duplicados.** Si no, los aproximados sobreviven.
3. **Separá el vacío, el cero y el centinela** antes de contar faltantes. Solo el centinela se convierte en ausencia.
4. **Después de convertir fechas, mirá el mínimo y el máximo.** Una conversión que no falla puede estar mal igual.
5. **Un extremo se cruza con las otras columnas de su fila** antes de decidir. No se saca porque afea el gráfico.
6. **Si dos métodos discrepan, decidí vos** e informá cuál usaste y por qué.
7. **Codificá y escalá al final**, con el texto ya unificado y la escala de cada variable escrita.
8. **Graficá cada variable según su escala**, y la ordinal siempre en el orden de la escala.
9. **Anotá cada decisión:** qué hiciste, sobre cuántas filas, y por qué.

Los ocho puntos de la clase 2 diagnosticaban sin tocar el dato. Estos nueve lo modifican, y por eso el último es el que manda: la lista de anotaciones que deja es la justificación que se evalúa.

8. LOS EJEMPLOS EN GOOGLE COLAB

Casi todo lo que describe este apunte está también en un notebook, con el código ya escrito y listo para ejecutar: [03 limpieza y graficos.ipynb](#). El relevamiento de cuarenta filas se fabrica ahí mismo, así que no depende de ningún portal. Una sola celda pide internet, y está avisada.

Una celda del notebook falla a propósito: es la que intenta convertir las fechas declarando un formato que no es el de todas. Está avisada antes de llegar. Cuando aparezca el bloque rojo, seguí con la celda siguiente.

Los seis bloques que vas a ejecutar, y a qué sección de este apunte corresponde cada uno:

1. **El relevamiento completo, y el orden.** Las cuarenta filas, y por qué unificar el texto antes o después de buscar duplicados no da lo mismo. Secciones 1.3 y 2.1.
2. **Duplicados y conflicto.** El duplicado exacto, el id repetido y el paso de cuarenta a treinta y siete. Sección 3.
3. **Las tres formas de que no haya un dato.** El vacío, el cero y el centinela, y qué le hace cada uno al promedio. Sección 4.1.
4. **Texto y fechas.** Los tres pasos mecánicos, y la conversión de fechas que no falla y corrompe dos filas. Secciones 5.1 y 5.2.
5. **Extremos, codificación y escalado.** El IQR contra el z-score, y las dos formas de codificar según la escala. Secciones 5.4, 6.1 y 6.2.
6. **Los cuatro gráficos.** Dos de barras, un histograma y una dispersión, cada uno sobre la variable que le corresponde. Sección 7.

Quedan afuera del notebook, porque se entienden leyendo, la clasificación de los tres mecanismos de faltantes de la sección 4.3, la normalización como estructura de la sección 6.3 y la apertura de la columna servicios de la sección 6.4.